

Análise do Projeto Agendamento

1. Visão Geral do Projeto

O projeto `agendamento` é uma aplicação Spring Boot desenvolvida em Java 21, utilizando Maven como ferramenta de build. Pelo nome e estrutura inicial, parece ser um sistema de agendamento, provavelmente uma API RESTful, conforme indicado pela dependência `spring-boot-starter-web`.

2. Dependências e Tecnologias Principais (pom.xml)

Com base no `pom.xml`, as principais tecnologias e dependências utilizadas são:

- **Spring Boot (versão 3.5.3):** Framework principal para construção da aplicação.
- **Java 21:** Versão da linguagem Java utilizada.
- **Maven:** Ferramenta de gerenciamento de dependências e build.
- **Spring Data JPA:** Para persistência de dados e interação com o banco de dados, utilizando o Hibernate como ORM padrão.
- **Spring Boot Starter Web:** Para construção de aplicações web, incluindo APIs RESTful.
- **Spring Boot Starter Validation:** Para validação de dados de entrada.
- **PostgreSQL:** Banco de dados relacional (em tempo de execução).
- **H2 Database:** Banco de dados em memória, provavelmente para testes ou desenvolvimento local.
- **MapStruct (versão 1.6.3):** Biblioteca para mapeamento de objetos (DTOs para entidades e vice-versa), gerando código em tempo de compilação.
- **Lombok (versão 1.18.32):** Biblioteca que reduz a verbosidade do código Java, gerando automaticamente getters, setters, construtores, etc.
- **Mockito:** Framework para criação de mocks em testes unitários.

3. Estrutura de Pacotes (src/main/java/br/com/sasaki/solution/agendamento)

A estrutura de pacotes segue um padrão comum de aplicações Spring Boot, indicando uma arquitetura em camadas:

- `controller/` : Provavelmente contém os endpoints REST que expõem a API.
- `dto/` : Contém os Data Transfer Objects, utilizados para transferir dados entre as camadas e para a comunicação com o cliente.
- `exception/` : Para classes de exceção personalizadas e tratamento de erros.
- `mapper/` : Contém as interfaces MapStruct para mapeamento de objetos.
- `model/` : Contém as entidades de domínio, que representam as tabelas do banco de dados.
- `repository/` : Contém as interfaces Spring Data JPA para acesso a dados.
- `service/` : Contém a lógica de negócio da aplicação.

4. Próximos Passos para Análise Detalhada

Para uma avaliação mais aprofundada da arquitetura e identificação de pontos de melhoria, os próximos passos serão:

1. **Analisar as classes de modelo (`model/`)**: Entender as entidades e seus relacionamentos.
2. **Analisar as interfaces de repositório (`repository/`)**: Verificar as operações de persistência definidas.
3. **Analisar as classes de serviço (`service/`)**: Compreender a lógica de negócio e a orquestração das operações.
4. **Analisar as classes de controlador (`controller/`)**: Entender os endpoints expostos e como eles interagem com a camada de serviço.

5. **Analisar os DTOs (`dto/`) e Mappers (`mapper/`):** Verificar como os dados são transferidos e mapeados.
6. **Analisar as classes de exceção (`exception/`):** Entender o tratamento de erros.
7. **Verificar a configuração do Spring Boot:** Arquivos `application.properties` ou `application.yml` para configurações de banco de dados, etc.
8. **Compilar e, se possível, executar o projeto:** Para verificar se há erros de compilação e observar o comportamento em tempo de execução.

Esta análise inicial servirá como base para identificar oportunidades de aplicação de Clean Code, princípios SOLID e outras boas práticas de desenvolvimento Java com Spring Boot.

5. Análise das Classes de Modelo (`model/`)

As classes de modelo representam as entidades do domínio e são mapeadas para tabelas no banco de dados. Foram analisadas as seguintes entidades:

5.1. `Cliente.java`

Representa a entidade `Cliente` com os seguintes atributos:

- `Id` (Long): Chave primária, gerada automaticamente.
- `nome` (String): Nome do cliente, não nulo.
- `cpfCnpj` (String): CPF ou CNPJ do cliente, não nulo e único.
- `telefone` (String): Telefone do cliente, não nulo.
- `email` (String): Email do cliente.

A classe utiliza anotações Lombok (`@Getter` , `@Setter` , `@NoArgsConstructor` , `@AllArgsConstructor` , `@Builder`) para reduzir a verbosidade, o que é uma boa prática. As anotações JPA (`@Entity` , `@Table` , `@Id` , `@GeneratedValue` , `@Column`) estão corretamente aplicadas para o mapeamento objeto-relacional.

5.2. Especialidade.java

Representa a entidade `Especialidade` com os seguintes atributos:

- `id` (Long): Chave primária, gerada automaticamente.
- `nomeEspecialidade` (String): Nome da especialidade, não nulo e mapeado para a coluna `nome_especialidade`.

Similar à classe `Cliente`, `Especialidade` também faz uso de Lombok e anotações JPA, seguindo um padrão consistente.

5.3. Profissional.java

Representa a entidade `Profissional` com os seguintes atributos:

- `id` (Long): Chave primária, gerada automaticamente.
- `nomeProfissional` (String): Nome do profissional, não nulo e mapeado para a coluna `nome_profissional`.
- `crm` (String): CRM do profissional, único e não nulo.
- `idCliente` (Cliente): Relacionamento `ManyToOne` com a entidade `Cliente`, mapeado para a coluna `cliente_id`.
- `idEspecialidade` (Especialidade): Relacionamento `ManyToOne` com a entidade `Especialidade`, mapeado para a coluna `especialidade_id`.

Observação: O relacionamento `ManyToOne` de `Profissional` com `Cliente` (`idCliente`) parece conceitualmente incorreto para um sistema de agendamento de serviços. Geralmente, um profissional atende *muitos* clientes, e não pertence a um único cliente. Se a intenção é que um profissional tenha uma lista de clientes que ele atendeu, o relacionamento deveria ser `ManyToMany` ou `OneToMany` (se o `Cliente` tiver uma lista de profissionais que o atenderam). Se a intenção é que o profissional seja um *usuário* do sistema que também é um cliente, talvez a modelagem precise ser revista para separar as responsabilidades de `Profissional` e `Cliente` como entidades distintas, ou ter uma entidade `Usuario` que possa ser tanto `Cliente` quanto `Profissional` através de papéis ou herança.

Este é um ponto importante para ser revisado na fase de refatoração, visando a clareza do domínio e a aplicação dos princípios SOLID (especialmente o Princípio da Responsabilidade Única - SRP).

6. Próximos Passos

Continuarei a análise das demais camadas do projeto, começando pelas interfaces de repositório.

7. Análise das Interfaces de Repositório (repository/)

As interfaces de repositório utilizam o Spring Data JPA, o que simplifica bastante a camada de persistência, fornecendo métodos CRUD (Create, Read, Update, Delete) prontos para uso e a capacidade de definir consultas personalizadas através de nomes de métodos.

7.1. ClienteRepository.java

Estende `JpaRepository<Cliente, Long>`, o que já oferece operações básicas para a entidade `Cliente`. Além disso, possui dois métodos de consulta personalizados:

- `existsByCpfCnpj(String cpfCnpj)` : Verifica se já existe um cliente com o CPF/CNPJ informado. Útil para validações de unicidade.
- `existsByEmail(String email)` : Verifica se já existe um cliente com o email informado. Também útil para validações de unicidade.

Esses métodos são bem definidos e seguem o padrão de nomeação do Spring Data JPA, o que é uma boa prática.

7.2. EspecialidadeRepository.java

Estende `JpaRepository<Especialidade, Long>`, fornecendo as operações básicas para a entidade `Especialidade`. Não há métodos de consulta personalizados, o que indica que as operações padrão são suficientes para esta entidade no momento.

7.3. ProfissionalRepository.java

Estende `JpaRepository<Profissional, Long>` , oferecendo as operações básicas para a entidade `Profissional` . Possui os seguintes métodos de consulta personalizados:

- `findAllByNomeProfissionalContainingIgnoreCase(String nomeProfissional, Pageable pageable)` : Permite buscar profissionais por parte do nome (ignorando maiúsculas/minúsculas) e com paginação. Isso é excelente para funcionalidades de busca e listagem.
- `existsByCrm(String crm)` : Verifica a unicidade do CRM do profissional.

7.4. Observações e Boas Práticas na Camada de Repositório

- **Uso do Spring Data JPA:** A utilização do Spring Data JPA é uma excelente prática, pois abstrai a complexidade do JDBC e do JPA, permitindo que o desenvolvedor se concentre na lógica de negócio. A definição de métodos de consulta por nome é poderosa e legível.
- **Unicidade:** Os métodos `existsBy...` são adequados para verificar a unicidade de campos antes de operações de persistência, evitando exceções de banco de dados e fornecendo feedback mais amigável ao usuário.
- **Paginação:** O uso de `Pageable` em `ProfissionalRepository` é crucial para aplicações que lidam com grandes volumes de dados, garantindo performance e escalabilidade ao evitar o carregamento de todos os registros de uma vez.

Ponto de Atenção: A modelagem do relacionamento `Profissional` com `Cliente` (`idCliente`) mencionada na análise das classes de modelo pode impactar as consultas futuras nesta camada. Se a revisão da modelagem for necessária, as interfaces de repositório também precisarão ser ajustadas para refletir os novos relacionamentos.

8. Próximos Passos

O próximo passo será analisar a camada de serviço (`service/`), que contém a lógica de negócio da aplicação. Em seguida, as camadas de DTO, Mapper e Controller serão examinadas.

9. Análise da Camada de Serviço (`service/`)

A camada de serviço é responsável por encapsular a lógica de negócio da aplicação, orquestrando as operações entre as camadas de repositório e de apresentação (controladores). O projeto segue o padrão de interface e implementação (`ClienteService` e `ClienteServiceImpl`), o que é uma boa prática para promover a coesão e o baixo acoplamento, facilitando a testabilidade e a manutenção.

9.1. `ClienteService.java` (Interface)

A interface `ClienteService` define o contrato para as operações relacionadas à entidade `Cliente`. Os métodos expostos são:

- `create(ClienteRequestDTO dto)` : Cria um novo cliente a partir de um DTO de requisição e retorna um DTO de resposta.
- `findById(Long id)` : Busca um cliente por ID e retorna um DTO de resposta.
- `findAll(Pageable pageable)` : Retorna uma página de clientes (DTOs de resposta) com suporte a paginação.
- `update(Long id, ClienteRequestDTO dto)` : Atualiza um cliente existente por ID com dados de um DTO de requisição e retorna um DTO de resposta.
- `delete(Long id)` : Exclui um cliente por ID.
- `findClienteByIdOrThrow(Long id)` : Método auxiliar para buscar um cliente por ID ou lançar uma exceção caso não seja encontrado. Este método é `public` na interface, mas sua implementação é `private` na classe de serviço, o que pode ser um ponto de atenção para a visibilidade e o contrato da interface.

9.2. `ClienteServiceImpl.java` (Implementação)

A classe `ClienteServiceImpl` implementa a interface `ClienteService` e contém a lógica de negócio para as operações de cliente. Utiliza injeção de dependência para o `ClienteRepository` e o `ClienteMapper` (embora o `ClienteMapper` seja estático, o que pode ser um ponto de refatoração).

Principais pontos da implementação:

- **Criação (`create`):**

- Chama `validarCnpjEmail(dto)` para verificar a unicidade do CPF/CNPJ e e-mail antes de salvar.
- Converte o DTO para entidade (`ClienteMapper.toEntity`).
- Salva a entidade no repositório (`repository.save`).
- Converte a entidade salva de volta para DTO de resposta (`ClienteMapper.toResponseDTO`).

- **Busca por ID (`findById`):**

- Utiliza `findClienteByIdOrThrow` para buscar o cliente e lidar com a ausência.
- Converte a entidade encontrada para DTO de resposta.

- **Busca todos (`findAll`):**

- Delega a paginação ao repositório (`repository.findAll(pageable)`).
- Mapeia cada entidade na página para um DTO de resposta (`.map(ClienteMapper::toResponseDTO)`).

- **Atualização (`update`):**

- Busca o cliente existente usando `findClienteByIdOrThrow` .
- Atualiza a entidade existente com os dados do DTO (`ClienteMapper.updateEntity`).
- Salva a entidade atualizada.
- Converte a entidade atualizada para DTO de resposta.

- **Exclusão (`delete`):**

- Busca o cliente usando `findClienteByIdOrThrow` .
- Exclui a entidade do repositório.

- **Método Auxiliar (`findClienteByIdOrThrow`):**

- Busca o cliente por ID e, se não encontrar, lança uma `BusinessException` com uma mensagem clara. O uso de `BusinessException` é adequado para erros de negócio.
- **Validações de Unicidade** (`validarCnpjEmail` , `validarSeCnpjJaExiste` , `validarSeEmailJaExiste`):
 - Esses métodos privados encapsulam a lógica de validação de unicidade, utilizando os métodos `existsBy...` do repositório. Lançam `BusinessException` em caso de violação.

9.3. Boas Práticas e Pontos de Atenção na Camada de Serviço

- **Separação de Responsabilidades (SRP):** A camada de serviço está bem definida para a lógica de negócio do `Cliente` . No entanto, a validação de unicidade poderia ser movida para uma camada de validação mais genérica ou para o próprio DTO (com anotações de validação customizadas), ou até mesmo para um serviço de domínio específico para validações, para manter o serviço mais focado nas regras de negócio principais.
- **Tratamento de Exceções:** O uso de `BusinessException` é apropriado para erros de negócio, permitindo um tratamento mais granular na camada de apresentação. O uso de `ResourceNotFoundException` (importado, mas não usado diretamente, apenas `BusinessException` é lançada) pode indicar uma inconsistência ou uma intenção de ter exceções mais específicas para recursos não encontrados.
- **Mapeamento de Objetos:** O uso do `MapStruct` é uma excelente escolha para o mapeamento, pois gera código otimizado e seguro em tempo de compilação. No entanto, a forma como `ClienteMapper` é chamado estaticamente (`ClienteMapper.toEntity`) pode dificultar a testabilidade se o mapper tiver dependências complexas no futuro. Injetar o mapper (se ele fosse um componente Spring) seria uma alternativa mais flexível, embora para mappers simples, o uso estático seja comum.
- **Consistência de Exceções:** A `findClienteByIdOrThrow` lança `BusinessException` , mas a `ResourceNotFoundException` está importada. É importante decidir qual exceção usar para recursos não encontrados e manter a consistência em todo o projeto. `ResourceNotFoundException` geralmente é mais semântica para este caso.

- **Relacionamento Profissional com Cliente** : A análise anterior sobre o relacionamento Profissional com Cliente é crucial aqui. Se a modelagem for alterada, a lógica de negócio no ClienteService (e potencialmente em outros serviços) precisará ser revisada para refletir as novas interações entre as entidades.

10. Próximos Passos

Continuarei a análise das demais classes de serviço (EspecialidadeService e ProfissionalService), e em seguida, as camadas de DTO, Mapper, Controller e Exception serão examinadas para uma visão completa da arquitetura.

11. Análise da Camada de Serviço (service/especialidade)

Seguindo o padrão de interface e implementação, a camada de serviço para Especialidade é composta por EspecialidadeService e EspecialidadeServiceImpl .

11.1. EspecialidadeService.java (Interface)

A interface EspecialidadeService define o contrato para as operações de Especialidade :

- criar(EspecialidadeRequestDTO dto) : Cria uma nova especialidade.
- listarTodas(Pageable pageable) : Lista todas as especialidades com paginação.
- buscarPorId(Long id) : Busca uma especialidade por ID.
- atualizar(Long id, EspecialidadeRequestDTO dto) : Atualiza uma especialidade existente.
- deletar(Long id) : Deleta uma especialidade.
- buscarEspecialidadePorId(Long id) : Método auxiliar para buscar a entidade Especialidade por ID ou lançar exceção.

11.2. EspecialidadeServiceImpl.java (Implementação)

A classe EspecialidadeServiceImpl implementa a interface EspecialidadeService e contém a lógica de negócio para Especialidade . Ela injeta EspecialidadeRepository e

EspecialidadeMapper .

Principais pontos da implementação:

- **Criação (`criar`)**: Converte DTO para entidade, salva no repositório e converte a entidade salva de volta para DTO de resposta.
- **Listagem (`listaTodas`)**: Delega a paginação ao repositório e mapeia as entidades para DTOs.
- **Busca por ID (`buscarPorId`)**: Utiliza o método auxiliar `buscarEspecialidadePorId` e mapeia para DTO.
- **Atualização (`atualizar`)**: Busca a entidade existente, atualiza seus campos com base no DTO usando o mapper (`mapper.updateEntityFromDTO`), salva e mapeia para DTO.
- **Exclusão (`deletar`)**: Busca a entidade e a deleta do repositório.
- **Método Auxiliar (`buscarEspecialidadePorId`)**: Busca a especialidade por ID e lança `ResourceNotFoundException` se não encontrar. Diferente de `ClienteServiceImpl` , que usou `BusinessException` para o mesmo propósito, aqui `ResourceNotFoundException` é usada, o que é mais semântico para indicar que um recurso não foi encontrado.

11.3. Boas Práticas e Pontos de Atenção na Camada de Serviço (Especialidade)

- **Consistência de Exceções**: O uso de `ResourceNotFoundException` aqui é uma boa prática para indicar que um recurso específico não foi encontrado. É importante manter essa consistência em todo o projeto, talvez refatorando `ClienteServiceImpl` para usar `ResourceNotFoundException` também, se for o caso de um recurso não encontrado.
- **Injeção de Mapper**: Diferente de `ClienteServiceImpl` que usou o `ClienteMapper` estaticamente, `EspecialidadeServiceImpl` injeta `EspecialidadeMapper` . A injeção é geralmente preferível para facilitar testes unitários e permitir maior flexibilidade (por exemplo, se o mapper precisar de outras dependências no futuro).
- **Nomenclatura de Métodos**: Os nomes dos métodos na interface e implementação (`criar` , `listaTodas` , `buscarPorId` , `atualizar` , `deletar`) estão em português, o que é

consistente com o restante do código. No entanto, é comum em projetos Java adotar a nomenclatura em inglês para métodos e variáveis, seguindo convenções da comunidade e frameworks. Isso não é um erro, mas uma questão de estilo e convenção que pode ser padronizada.

12. Próximos Passos

A próxima etapa é analisar a camada de serviço para `Profissional`, seguida pela análise dos DTOs, Mappers, Controllers e classes de exceção.

13. Análise da Camada de Serviço (`service/profissional`)

A camada de serviço para `Profissional` segue o mesmo padrão de interface e implementação, com `ProfissionalService` e `ProfissionalServiceImpl`.

13.1. `ProfissionalService.java` (Interface)

A interface `ProfissionalService` define o contrato para as operações de `Profissional`:

- `salvar(ProfissionalRequestDTO dto)` : Salva um novo profissional.
- `listarTodos(Pageable pageable)` : Lista todos os profissionais com paginação.
- `buscarPorId(Long id)` : Busca um profissional por ID.
- `buscarPorNome(String nomeProfissional, Pageable pageable)` : Busca profissionais por nome com paginação.
- `atualizar(Long id, ProfissionalRequestDTO dto)` : Atualiza um profissional existente.
- `deletar(Long id)` : Deleta um profissional.

13.2. `ProfissionalServiceImpl.java` (Implementação)

A classe `ProfissionalServiceImpl` implementa a interface `ProfissionalService` e contém a lógica de negócio para `Profissional`. Ela injeta `ProfissionalRepository`, `ClienteRepository`, `EspecialidadeRepository`, `ProfissionalMapper`, `ClienteService` e `EspecialidadeService`.

Principais pontos da implementação:

- **Criação (salvar):**
 - Chama `verificaSeExisteCrmCadastrado(dto.crm())` para validar a unicidade do CRM.
 - Converte o DTO para entidade (`mapper.toEntity(dto)`).
 - **Ponto Crítico:** Define `idCliente` e `idEspecialidade` do `Profissional` buscando as entidades `Cliente` e `Especialidade` através dos respectivos serviços (`clienteService.findClienteByIdOrThrow(dto.idCliente())` e `especialidadeService.buscarEspecialidadePorId(dto.idEspecialidade())`). Isso confirma a dependência do `Profissional` em `Cliente` e `Especialidade` no momento da criação/atualização.
 - Salva a entidade no repositório e converte para DTO de resposta.
- **Listagem (listarTodos):** Delega a paginação ao repositório e mapeia as entidades para DTOs.
- **Busca por ID (buscarPorId):** Busca o profissional por ID e, se não encontrar, lança `EntityNotFoundException`. Note que aqui é usada `EntityNotFoundException`, enquanto em `ClienteServiceImpl` foi `BusinessException` e em `EspecialidadeServiceImpl` foi `ResourceNotFoundException` para o mesmo cenário de recurso não encontrado. Isso é uma inconsistência que precisa ser padronizada.
- **Busca por Nome (buscarPorNome):** Utiliza o método de busca por nome do repositório com paginação.
- **Atualização (atualizar):**
 - Busca o profissional existente, lança `EntityNotFoundException` se não encontrar.
 - Atualiza os campos do profissional com o DTO usando o mapper (`mapper.updateEntityFromDTO`).
 - **Ponto Crítico:** Assim como na criação, atualiza `idCliente` e `idEspecialidade` buscando as entidades através dos serviços.

- Salva a entidade atualizada e converte para DTO.
- **Exclusão (deletar)**: Verifica a existência do profissional por ID e o deleta. Lança `EntityNotFoundException` se não encontrar.
- **Validação de CRM (verificaSeExisteCrmCadastrado)**: Encapsula a lógica de validação de unicidade do CRM, lançando `BusinessException` em caso de CRM duplicado.

13.3. Boas Práticas e Pontos de Atenção na Camada de Serviço (Profissional)

- **Injeção de Dependências**: A injeção de `ClienteService` e `EspecialidadeService` em `ProfissionalServiceImpl` é uma boa prática, permitindo a reutilização da lógica de negócio e o baixo acoplamento entre os serviços.
- **Consistência de Exceções (Crítico)**: A inconsistência no tipo de exceção lançada para recursos não encontrados (`BusinessException` , `ResourceNotFoundException` , `EntityNotFoundException`) é um ponto de melhoria significativo. É fundamental padronizar o tratamento de exceções para melhorar a clareza do código, a manutenção e a experiência do desenvolvedor que consumirá a API.
- **Modelagem do Relacionamento Profissional com Cliente** : Este é o ponto mais crítico identificado até agora. A forma como `Profissional` está relacionado a `Cliente` (`@ManyToOne idCliente`) e como essa relação é tratada na camada de serviço (`profissional.setIdCliente(clienteService.findClienteByIdOrThrow(dto.idCliente()))`) sugere que um profissional está associado a *um* cliente específico. Em um sistema de agendamento, o mais comum é que um profissional atenda *vários* clientes. Se a intenção é que o `Profissional` seja um *usuário* do sistema que também pode ser um `Cliente` , a modelagem de `Usuario` com papéis ou herança seria mais apropriada. Se a intenção é que `idCliente` represente o *proprietário* do estabelecimento ou algo similar, o nome da propriedade e o relacionamento precisam ser mais claros. **Esta modelagem atual viola o Princípio da Responsabilidade Única (SRP) e o Princípio Aberto/Fechado (OCP) se a intenção for estender a funcionalidade de um profissional para atender múltiplos clientes sem modificar a entidade Profissional diretamente.** Será necessário discutir isso com o usuário para entender a intenção original e propor uma refatoração adequada.

- **Nomenclatura de Métodos:** Assim como em `EspecialidadeService`, a nomenclatura em português (`salvar`, `listarTodos`, etc.) é consistente, mas a padronização para inglês é uma convenção comum em projetos Java.

14. Próximos Passos

Com a análise da camada de serviço concluída, os próximos passos serão:

1. **Analisar os DTOs (`dto/`):** Entender a estrutura dos objetos de transferência de dados.
2. **Analisar os Mappers (`mapper/`):** Verificar como os objetos são mapeados entre entidades e DTOs.
3. **Analisar os Controladores (`controller/`):** Entender os endpoints da API e como eles interagem com a camada de serviço.
4. **Analisar as classes de exceção (`exception/`):** Detalhar o tratamento de erros e propor padronização.
5. **Revisar a modelagem de `Profissional` e `Cliente` :** Este ponto crítico será abordado com o usuário para definir a melhor abordagem.
6. **Compilar e, se possível, executar o projeto:** Para verificar se há erros de compilação e observar o comportamento em tempo de execução.

15. Análise dos DTOs (`dto/cliente`)

Os Data Transfer Objects (DTOs) são utilizados para transferir dados entre as camadas da aplicação, desacoplando as entidades de domínio da camada de apresentação e evitando a exposição direta dos modelos de banco de dados. O projeto utiliza records do Java 16+, o que é uma excelente prática para DTOs, pois são imutáveis e concisos.

15.1. `ClienteRequestDTO.java`

Este DTO é utilizado para receber dados de requisições relacionadas a `Cliente` (criação e atualização). Possui os seguintes campos:

- `nome` (String): Anotado com `@NotBlank(message = "Nome é obrigatório.")` , garantindo que o nome não seja nulo ou vazio.
- `cpfCnpj` (String): Anotado com `@NotBlank(message = "CNPJ é obrigatório.")` , garantindo que o CPF/CNPJ não seja nulo ou vazio.
- `telefone` (String): Anotado com `@NotBlank(message = "Telefone é obrigatório.")` , garantindo que o telefone não seja nulo ou vazio.
- `email` (String): Anotado com `@NotBlank(message = "E-mail é obrigatório.")` e `@Email(message = "Formato de e-mail inválido.")` , garantindo que o e-mail não seja nulo/vazio e que tenha um formato válido.

Boas Práticas e Pontos de Atenção:

- **Uso de Records:** A utilização de records para DTOs é uma ótima prática, pois eles são ideais para classes que servem principalmente para transportar dados. São imutáveis por padrão, o que contribui para a segurança e previsibilidade do código.
- **Validações Declarativas:** O uso de anotações de validação (`@NotBlank` , `@Email`) do Bean Validation (Jakarta Validation) é excelente. Isso centraliza as regras de validação e as torna declarativas, facilitando a leitura e manutenção. Essas validações são aplicadas automaticamente pelo Spring Boot quando o DTO é recebido em um controlador.
- **Mensagens de Erro:** As mensagens de erro personalizadas (``message =`

`"Nome é obrigatório."``) são úteis para fornecer feedback claro ao usuário em caso de erros de validação.

15.2. `ClienteResponseDTO.java`

Este DTO é utilizado para retornar dados de `Cliente` em respostas de requisições. Possui os seguintes campos:

- `id` (Long)
- `nome` (String)
- `cpfCnpj` (String)

- `telefone` (String)
- `email` (String)

Boas Práticas e Pontos de Atenção:

- **Uso de Records:** Assim como o `RequestDTO`, o `ResponseDTO` também se beneficia do uso de records, sendo conciso e imutável.
- **Exposição de Dados:** O `ResponseDTO` expõe apenas os dados necessários para o cliente, evitando a exposição de informações sensíveis ou internas da entidade.

16. Análise dos DTOs (`dto/especialidade`)

16.1. `EspecialidadeRequestDTO.java`

Este DTO é utilizado para receber dados de requisições relacionadas a `Especialidade`. Possui o campo:

- `nomeEspecialidade` (String): Anotado com `@NotBlank(message = "Nome da especialidade é obrigatório.")`.

16.2. `EspecialidadeResponseDTO.java`

Este DTO é utilizado para retornar dados de `Especialidade` em respostas. Possui os campos:

- `id` (Long)
- `nomeEspecialidade` (String)

17. Análise dos DTOs (`dto/profissional`)

17.1. `ProfissionalRequestDTO.java`

Este DTO é utilizado para receber dados de requisições relacionadas a `Profissional`. Possui os campos:

- `nomeProfissional` (String): Anotado com `@NotBlank(message = "Nome do profissional é obrigatório.")` .
- `crm` (String): Anotado com `@NotBlank(message = "CRM é obrigatório.")` .
- `idCliente` (Long): ID do cliente associado ao profissional. Anotado com `@NotNull(message = "ID do cliente é obrigatório.")` .
- `idEspecialidade` (Long): ID da especialidade associada ao profissional. Anotado com `@NotNull(message = "ID da especialidade é obrigatório.")` .

Ponto de Atenção: A presença de `idCliente` neste DTO reforça a preocupação com a modelagem do relacionamento `Profissional` com `Cliente` . Se um profissional pode atender múltiplos clientes, este campo não deveria estar aqui, ou a modelagem deveria ser outra (ex: uma lista de IDs de clientes, ou um relacionamento `ManyToMany`).

17.2. `ProfessionalResponseDTO.java`

Este DTO é utilizado para retornar dados de `Profissional` em respostas. Possui os campos:

- `id` (Long)
- `nomeProfissional` (String)
- `crm` (String)
- `idCliente` (Long): ID do cliente associado.
- `idEspecialidade` (Long): ID da especialidade associada.

Ponto de Atenção: Similar ao `RequestDTO` , a inclusão de `idCliente` no `ResponseDTO` para `Profissional` merece atenção devido à modelagem do relacionamento.

18. Boas Práticas e Pontos de Atenção na Camada de DTOs

- **Records para DTOs:** Excelente escolha para concisão e imutabilidade.
- **Validações Declarativas:** O uso de anotações de validação é fundamental para garantir a integridade dos dados de entrada.

- **Separação Request/Response:** A distinção entre DTOs de requisição e resposta é uma boa prática, permitindo que cada um contenha apenas os campos relevantes para sua finalidade.
- **Modelagem de Relacionamentos em DTOs:** A forma como os relacionamentos são representados nos DTOs (apenas IDs) é adequada para a comunicação entre camadas. No entanto, a semântica do relacionamento `Profissional - Cliente` ainda é um ponto a ser esclarecido e possivelmente refatorado.

19. Próximos Passos

Com a análise dos DTOs concluída, os próximos passos serão:

1. **Analisar os Mappers (`mapper/`):** Verificar como os objetos são mapeados entre entidades e DTOs.
2. **Analisar os Controladores (`controller/`):** Entender os endpoints da API e como eles interagem com a camada de serviço.
3. **Analisar as classes de exceção (`exception/`):** Detalhar o tratamento de erros e propor padronização.
4. **Revisar a modelagem de `Profissional` e `Cliente` :** Este ponto crítico será abordado com o usuário para definir a melhor abordagem.
5. **Compilar e, se possível, executar o projeto:** Para verificar se há erros de compilação e observar o comportamento em tempo de execução.

20. Análise dos Mappers (`mapper/`)

A camada de mappers é responsável por converter objetos entre diferentes representações, como entidades de domínio e DTOs. O projeto utiliza o MapStruct, uma biblioteca que gera código de mapeamento em tempo de compilação, o que é eficiente e seguro. A estrutura inclui uma interface `BaseMapper` e implementações específicas para cada entidade.

20.1. `BaseMapper.java`

Esta interface genérica define os métodos comuns de mapeamento que todas as interfaces de mapper devem implementar:

- `toEntity(Req dto)` : Converte um DTO de requisição para uma entidade.
- `toDTO(E entity)` : Converte uma entidade para um DTO de resposta.
- `toDTOList(List<E> entityList)` : Converte uma lista de entidades para uma lista de DTOs de resposta.
- `updateEntityFromDTO(Req dto, @MappingTarget E entity)` : Atualiza uma entidade existente com base em um DTO de requisição, ignorando propriedades nulas. A anotação `@BeanMapping(nullValuePropertyMappingStrategy = NullValuePropertyMappingStrategy.IGNORE)` é uma boa prática para atualizações parciais.

Boas Práticas: O uso de uma interface base para mappers é uma excelente prática, pois promove a reutilização de código e a padronização dos métodos de mapeamento.

20.2. `ClienteMapper.java`

Esta classe é uma implementação manual dos métodos de mapeamento para a entidade `Cliente`. Ela contém métodos estáticos:

- `toEntity(ClienteRequestDTO dto)` : Converte `ClienteRequestDTO` para `Cliente` usando o `Builder` do Lombok.
- `toResponseDTO(Cliente cliente)` : Converte `Cliente` para `ClienteResponseDTO`.
- `updateEntity(Cliente cliente, ClienteRequestDTO dto)` : Atualiza uma instância de `Cliente` com dados de `ClienteRequestDTO`.

Ponto de Atenção: Diferente dos outros mappers que utilizam a anotação `@Mapper` do MapStruct e estendem `BaseMapper`, `ClienteMapper` é uma classe manual com métodos estáticos. Isso gera uma inconsistência no padrão de mapeamento do projeto. Embora funcione, o ideal seria que `ClienteMapper` também fosse uma interface anotada com `@Mapper` e estendesse `BaseMapper`, permitindo que o MapStruct gerasse a implementação automaticamente. Isso traria os benefícios de segurança de tipo, desempenho e consistência com o restante do projeto.

20.3. EspecialidadeMapper.java

Esta é uma interface que estende `BaseMapper<Especialidade, EspecialidadeRequestDTO, EspecialidadeResponseDTO>` e é anotada com `@Mapper(componentModel = "spring")`. Isso indica que o MapStruct irá gerar uma implementação Spring Bean para esta interface, que poderá ser injetada em outras classes (como visto em `EspecialidadeServiceImpl`).

Boas Práticas: Esta é a forma recomendada de usar o MapStruct, aproveitando a geração automática de código e a integração com o Spring.

21. Próximos Passos

Continuarei a análise dos mappers, lendo `ProfissionalMapper.java` e `ReferenceMapper.java`, e em seguida, as camadas de Controller e Exception serão examinadas. O ponto de inconsistência no `ClienteMapper` será um dos focos para refatoração.

20.4. ProfissionalMapper.java

Esta é uma interface que estende `BaseMapper<Profissional, ProfissionalRequestDTO, ProfissionalResponseDTO>` e é anotada com `@Mapper(componentModel = "spring", uses = ReferenceMapper.class)`. A inclusão de `uses = ReferenceMapper.class` indica que o MapStruct deve utilizar os métodos definidos em `ReferenceMapper` para mapear as entidades `Cliente` e `Especialidade` (que são referenciadas por ID nos DTOs de `Profissional`).

Boas Práticas: Esta é a forma correta de lidar com mapeamentos de relacionamentos em DTOs quando apenas os IDs são expostos, permitindo que o MapStruct resolva as referências automaticamente.

20.5. ReferenceMapper.java

Esta classe é um componente Spring (`@Component`) que fornece métodos auxiliares para mapear IDs para entidades de referência e vice-versa. Isso é particularmente útil para o MapStruct quando ele precisa converter um `Long` (ID) para uma entidade (`Cliente` ou `Especialidade`) e vice-versa, sem precisar carregar a entidade completa do banco de dados para o mapeamento.

- `toCliente(Long id)` : Cria uma instância de `Cliente` com o ID fornecido. **Ponto de Atenção:** Esta abordagem cria uma entidade

“proxy” com apenas o ID. O Hibernate/JPA entende que esta é uma referência a uma entidade existente e, ao salvar a entidade principal (como `Profissional`), ele apenas inserirá o ID na chave estrangeira, sem precisar fazer um `SELECT` para buscar o `Cliente` ou a `Especialidade` completos. Isso é eficiente, mas assume que o ID fornecido é válido. A validação da existência da entidade é feita na camada de serviço, o que é correto.

- `toEspecialidade(Long id)` : Similar ao `toCliente`, cria uma instância de `Especialidade` com o ID fornecido.
- `fromCliente(Cliente cliente)` : Extrai o ID de uma entidade `Cliente`.
- `fromEspecialidade(Especialidade especialidade)` : Extrai o ID de uma entidade `Especialidade`.

Boas Práticas: O uso de um `ReferenceMapper` é uma técnica avançada e eficiente para lidar com relacionamentos em DTOs, evitando consultas desnecessárias ao banco de dados durante o mapeamento.

22. Próximos Passos

Com a análise dos mappers concluída, os próximos passos são:

1. **Analisar os Controladores (`controller/`):** Entender os endpoints da API e como eles interagem com a camada de serviço.
2. **Analisar as classes de exceção (`exception/`):** Detalhar o tratamento de erros e propor padronização.
3. **Revisar a modelagem de `Profissional` e `Cliente` :** Este ponto crítico será abordado com o usuário para definir a melhor abordagem.
4. **Compilar e, se possível, executar o projeto:** Para verificar se há erros de compilação e observar o comportamento em tempo de execução.

23. Análise dos Controladores (`controller/`)

A camada de controladores é a porta de entrada da aplicação, responsável por receber as requisições HTTP, delegar a lógica de negócio para a camada de serviço e retornar as respostas HTTP. O projeto utiliza o Spring Web para construir APIs RESTful.

23.1. `ClienteController.java`

Este controlador é responsável por gerenciar as operações CRUD para a entidade `Cliente`. É anotado com `@RestController` (que combina `@Controller` e `@ResponseBody`) e `@RequestMapping("api/clientes")`, definindo o endpoint base para os recursos de cliente. A injeção de dependência do `ClienteService` é feita via `@RequiredArgsConstructor` do Lombok.

Endpoints e Operações:

- **`POST /api/clientes` (create):**
 - Recebe um `ClienteRequestDTO` no corpo da requisição (`@RequestBody`).
 - A anotação `@Valid` garante que as validações definidas no DTO sejam aplicadas automaticamente pelo Bean Validation.
 - Delega a criação para `clienteService.create(dto)`.
 - Retorna `ResponseEntity.status(HttpStatus.CREATED).body(...)`, indicando sucesso na criação (código HTTP 201).
- **`GET /api/clientes/{id}` (findById):**
 - Recebe o ID do cliente como parte da URL (`@PathVariable Long id`).
 - Delega a busca para `clienteService.findById(id)`.
 - Retorna `ResponseEntity.ok(...)`, com o DTO do cliente encontrado (código HTTP 200).
- **`GET /api/clientes` (findAll):**
 - Recebe um objeto `Pageable` como parâmetro, permitindo paginação e ordenação automáticas fornecidas pelo Spring Data.
 - Delega a busca paginada para `clienteService.findAll(pageable)`.

- Retorna `ResponseEntity.ok(...)` , com uma página de DTOs de cliente (código HTTP 200).
- **PUT /api/clientes/{id} (update):**
 - Recebe o ID do cliente na URL e o `ClienteRequestDTO` no corpo da requisição.
 - A anotação `@Valid` também é aplicada aqui.
 - Delega a atualização para `clienteService.update(id, dto)` .
 - Retorna `ResponseEntity.ok(...)` , com o DTO do cliente atualizado (código HTTP 200).
- **DELETE /api/clientes/{id} (delete):**
 - Recebe o ID do cliente na URL.
 - Delega a exclusão para `clienteService.delete(id)` .
 - Retorna `ResponseEntity.noContent().build()` , indicando sucesso na exclusão sem conteúdo de retorno (código HTTP 204).

23.2. Boas Práticas e Pontos de Atenção na Camada de Controladores (Cliente)

- **RESTful Design:** O controlador segue os princípios RESTful, utilizando os verbos HTTP (`POST` , `GET` , `PUT` , `DELETE`) de forma semântica para as operações CRUD.
- **Separação de Responsabilidades:** O controlador atua como uma camada fina, delegando a lógica de negócio para o `ClienteService` , o que é uma boa prática e segue o Princípio da Responsabilidade Única (SRP).
- **Validação Automática:** O uso de `@Valid` nos DTOs de requisição é excelente para garantir que os dados de entrada sejam válidos antes de serem processados pela lógica de negócio.
- **Tratamento de Exceções:** Embora não explicitamente visível neste arquivo, o tratamento de exceções (como `BusinessException` ou `ResourceNotFoundException` lançadas na camada de serviço) deve ser centralizado em um `@ControllerAdvice` ou

`@RestControllerAdvice` para fornecer respostas de erro padronizadas e amigáveis ao cliente da API. A inconsistência no tipo de exceção lançada para recursos não encontrados (`BusinessException` em `ClienteServiceImpl` vs. `ResourceNotFoundException` em `EspecialidadeServiceImpl` vs. `EntityNotFoundException` em `ProfissionalServiceImpl`) é um ponto crítico que afetará o tratamento de erros nesta camada.

- **Paginação:** A utilização de `Pageable` nos métodos `GET` é uma ótima prática para lidar com grandes volumes de dados de forma eficiente.

24. Próximos Passos

Continuarei a análise dos controladores, lendo `EspecialidadeController.java` e `ProfissionalController.java`, e em seguida, as classes de exceção serão examinadas. A padronização do tratamento de exceções será um foco importante na fase de refatoração.

23.3. EspecialidadeController.java

Este controlador gerencia as operações CRUD para a entidade `Especialidade`. É anotado com `@RestController` e `@RequestMapping("api/especialidade")`. A injeção de dependência do `EspecialidadeService` é feita via `@RequiredArgsConstructor`.

Endpoints e Operações:

- **POST /api/especialidade (criar):**
 - Recebe um `EspecialidadeRequestDTO` com `@RequestBody` e `@Valid`.
 - Delega a criação para `especialidadeService.criar(dto)`.
 - Retorna `ResponseEntity.ok(...)`. **Ponto de Atenção:** Para operações de criação bem-sucedidas, o código de status HTTP `201 Created` é mais semântico do que `200 OK`. Isso deve ser padronizado.
- **GET /api/especialidade (listarTodas):**
 - Recebe `Pageable` com `@PageableDefault(size = 10, sort = "nomeEspecialidade")`, definindo um tamanho padrão de página e ordenação.

- Delega a busca paginada para `especialidadeService.listaTodas(pageable)` .
- Retorna `ResponseEntity.ok(...)` .
- **GET /api/especialidade/{id} (buscarPorId):**
 - Recebe o ID da especialidade com `@PathVariable` .
 - Delega a busca para `especialidadeService.buscarPorId(id)` .
 - Retorna `ResponseEntity.ok(...)` .
- **PUT /api/especialidade/{id} (atualizar):**
 - Recebe o ID na URL e o `EspecialidadeRequestDTO` no corpo, com `@Valid` .
 - Delega a atualização para `especialidadeService.atualizar(id, dto)` .
 - Retorna `ResponseEntity.ok(...)` .
- **DELETE /api/especialidade/{id} (deletar):**
 - Recebe o ID na URL.
 - Delega a exclusão para `especialidadeService.deletar(id)` .
 - Retorna `ResponseEntity.noContent().build()` , indicando sucesso na exclusão (código HTTP 204).

23.4. Boas Práticas e Pontos de Atenção na Camada de Controladores (Especialidade)

- **Consistência RESTful:** O controlador segue os princípios RESTful, mas a resposta `200 OK` para `POST` é uma inconsistência que deve ser corrigida para `201 Created` .
- **@PageableDefault** : O uso de `@PageableDefault` é uma boa prática para fornecer valores padrão para paginação, tornando a API mais robusta e fácil de usar.
- **Nomenclatura de Métodos:** Assim como na camada de serviço, os nomes dos métodos estão em português. A padronização para inglês é uma convenção comum em projetos Java.

25. Próximos Passos

Continuarei a análise dos controladores, lendo `ProfissionalController.java`, e em seguida, as classes de exceção serão examinadas. A padronização do tratamento de exceções e dos códigos de status HTTP será um foco importante na fase de refatoração.

23.5. `ProfissionalController.java`

Este controlador gerencia as operações CRUD para a entidade `Profissional`. É anotado com `@RestController` e `@RequestMapping("/api/profissionais")`. A injeção de dependência do `ProfissionalService` é feita via `@RequiredArgsConstructor`.

Endpoints e Operações:

- **`POST /api/profissionais` (criar):**
 - Recebe um `ProfissionalRequestDTO` com `@RequestBody` e `@Valid`.
 - Delega a criação para `profissionalService.salvar(dto)`.
 - Retorna `ResponseEntity.status(HttpStatus.CREATED).body(...)`, o que é o código de status HTTP correto para criação bem-sucedida (201 Created).
- **`GET /api/profissionais` (listar):**
 - Recebe `Pageable` para paginação e ordenação.
 - Delega a busca paginada para `profissionalService.listarTodos(pageable)`.
 - Retorna `ResponseEntity.ok(...)`.
- **`GET /api/profissionais/{id}` (buscarPorId):**
 - Recebe o ID do profissional com `@PathVariable`.
 - Delega a busca para `profissionalService.buscarPorId(id)`.
 - Retorna `ResponseEntity.ok(...)`.
- **`GET /api/profissionais/search` (buscarPorNome):**

- Recebe o nome do profissional como parâmetro de requisição (`@RequestParam String nome`) e `Pageable` .
- Delega a busca por nome paginada para `profissionalService.buscarPorNome(nome, pageable)` .
- Retorna `ResponseEntity.ok(...)` .
- **PUT /api/profissionais/{id} (atualizar):**
 - Recebe o ID na URL e o `ProfessionalRequestDTO` no corpo, com `@Valid` .
 - Delega a atualização para `profissionalService.atualizar(id, dto)` .
 - Retorna `ResponseEntity.ok(...)` .
- **DELETE /api/profissionais/{id} (deletar):**
 - Recebe o ID na URL.
 - Delega a exclusão para `profissionalService.deletar(id)` .
 - Retorna `ResponseEntity.noContent().build()` , indicando sucesso na exclusão (código HTTP 204).

23.6. Boas Práticas e Pontos de Atenção na Camada de Controladores (Professional)

- **Consistência RESTful:** Este controlador segue as boas práticas RESTful para os códigos de status HTTP, retornando `201 Created` para `POST` e `204 No Content` para `DELETE` .
- **Endpoint de Busca Específico:** A criação de um endpoint `/search` para buscar por nome (`/api/profissionais/search?nome=...`) é uma boa prática para consultas mais específicas, separando-as da listagem geral.
- **Modelagem do Relacionamento Profissional com Cliente :** Assim como nas camadas de modelo, repositório e serviço, a presença de `idCliente` no `ProfessionalRequestDTO` e `ProfessionalResponseDTO` e seu tratamento na camada de serviço são pontos críticos que merecem atenção especial. A forma como essa relação é tratada no `ProfessionalService`

(buscando a entidade `Cliente` por ID) é replicada aqui na camada de apresentação, o que é esperado, mas a modelagem subjacente ainda precisa ser revisada.

- **Nomenclatura de Métodos:** A nomenclatura em português é consistente com o restante do projeto.

26. Próximos Passos

Com a análise dos controladores concluída, os próximos passos serão:

1. **Analisar as classes de exceção (`exception/`):** Detalhar o tratamento de erros e propor padronização.
2. **Revisar a modelagem de `Profissional` e `Cliente` :** Este ponto crítico será abordado com o usuário para definir a melhor abordagem.
3. **Compilar e, se possível, executar o projeto:** Para verificar se há erros de compilação e observar o comportamento em tempo de execução.

27. Análise das Classes de Exceção (`exception/`)

A camada de exceções é responsável por definir exceções personalizadas e centralizar o tratamento de erros da aplicação, fornecendo respostas HTTP consistentes e significativas para o cliente da API.

27.1. `BusinessException.java`

Esta é uma exceção de tempo de execução (`RuntimeException`) personalizada para representar erros de regra de negócio. Por exemplo, tentar cadastrar um cliente com um CPF/CNPJ que já existe. Estender `RuntimeException` é uma prática comum para exceções que indicam um problema que não pode ser recuperado no mesmo contexto.

27.2. `ResourceNotFoundException.java`

Similar à `BusinessException` , esta é uma `RuntimeException` personalizada para indicar que um recurso específico não foi encontrado. É mais semântica do que uma `BusinessException`

genérica para este cenário.

27.3. GlobalExceptionHandler.java

Esta classe é anotada com `@RestControllerAdvice`, o que a torna um componente global para tratamento de exceções em todos os controladores (`@RestController`). Ela contém métodos anotados com `@ExceptionHandler` para lidar com tipos específicos de exceções:

- **`handleBusinessException(BusinessException ex)`** : Captura `BusinessException`, retorna HTTP 400 (Bad Request) com a mensagem da exceção.
- **`handleEntityNotFound(EntityNotFoundException ex)`** : Captura `EntityNotFoundException` (do `jakarta.persistence`), retorna HTTP 404 (Not Found) com a mensagem da exceção.
- **`handleValidationErrors(MethodArgumentNotValidException ex)`** : Captura `MethodArgumentNotValidException` (lançada quando a validação de `@Valid` falha), retorna HTTP 400 (Bad Request) com um mapa dos campos e suas respectivas mensagens de erro. Esta é uma excelente prática para fornecer feedback detalhado sobre erros de validação.
- **`handleGenericException(Exception ex)`** : Um handler de fallback que captura qualquer outra exceção não tratada, retornando HTTP 500 (Internal Server Error) com uma mensagem genérica. Isso evita que exceções inesperadas exponham detalhes internos da aplicação.

27.4. Boas Práticas e Pontos de Atenção na Camada de Exceções

- **Tratamento Centralizado:** O uso de `@RestControllerAdvice` é a melhor prática para centralizar o tratamento de exceções em aplicações Spring Boot, mantendo os controladores limpos e focados em sua responsabilidade principal.
- **Exceções Personalizadas:** A criação de exceções personalizadas como `BusinessException` e `ResourceNotFoundException` é uma boa prática para representar erros de domínio de forma clara.
- **Inconsistência no Uso de Exceções (Crítico):** Como mencionado anteriormente, a camada de serviço usa diferentes exceções para o mesmo cenário de “recurso não

encontrado” (`BusinessException` em `ClienteServiceImpl` , `ResourceNotFoundException` em `EspecialidadeServiceImpl` , e `EntityNotFoundException` em `ProfissionalServiceImpl`). O `GlobalExceptionHandler` trata `EntityNotFoundException` , mas não `ResourceNotFoundException` . Isso significa que, se `ResourceNotFoundException` for lançada, ela será capturada pelo `handleGenericException` e retornará um HTTP 500, o que não é o comportamento esperado (deveria ser 404). É crucial padronizar o uso de uma única exceção para “recurso não encontrado” (por exemplo, `ResourceNotFoundException`) e garantir que ela seja tratada corretamente no `GlobalExceptionHandler` .

28. Conclusão da Análise e Próximos Passos

A análise detalhada das camadas do projeto está concluída. A arquitetura geral é sólida e segue muitas boas práticas do Spring Boot. No entanto, foram identificados pontos críticos que precisam ser abordados para melhorar a qualidade, manutenibilidade e consistência do código.

Próximos Passos:

1. **Compilar o projeto:** Para garantir que não há erros de compilação.
2. **Apresentar a análise ao usuário:** Comunicar os pontos fortes e as oportunidades de melhoria identificadas.
3. **Discutir a modelagem de `Profissional` e `Cliente` :** Este é o ponto mais importante a ser esclarecido com o usuário para definir a refatoração correta.
4. **Propor um plano de refatoração:** Com base na discussão, criar um plano de ação para aplicar as melhorias, incluindo a padronização de exceções, a refatoração do `ClienteMapper` e a correção da modelagem de domínio.
5. **Implementar as melhorias:** Executar o plano de refatoração e, em seguida, prosseguir com o desenvolvimento de novas funcionalidades, se houver.

29. Análise da Falha na Compilação

Após a instalação do OpenJDK 21 e a configuração do `JAVA_HOME`, a compilação do projeto falhou durante a execução dos testes, especificamente no teste

`AgendamentoApplicationTests.contextLoads()`. A mensagem de erro `Failed to load ApplicationContext` indica que o contexto do Spring Boot não pôde ser carregado corretamente durante a execução dos testes.

Causas Comuns para `Failed to load ApplicationContext` em testes Spring Boot:

- **Configuração de Banco de Dados:** O projeto utiliza PostgreSQL para produção e H2 para desenvolvimento/testes. É comum que o contexto de teste tente se conectar a um banco de dados que não está disponível ou configurado corretamente para o ambiente de teste. Se o H2 não estiver configurado para ser usado em testes, ou se houver alguma configuração de banco de dados que exija um banco de dados externo, o contexto pode falhar ao iniciar.
- **Dependências Ausentes ou Conflitantes:** Embora o `pom.xml` pareça completo, pode haver alguma dependência transitiva faltando ou em conflito que impeça o carregamento do contexto.
- **Problemas na Classe Principal da Aplicação:** Erros na classe `AgendamentoApplication.java` ou em suas configurações podem impedir o carregamento do contexto.

30. Próximos Passos para Solução da Compilação

Para resolver o problema de compilação e continuar a análise, as seguintes ações serão tomadas:

1. **Verificar a configuração de testes:** Inspecionar o arquivo `application.properties` ou `application.yml` (se existirem) e as configurações de teste para garantir que o banco de dados H2 esteja configurado corretamente para o ambiente de teste.
2. **Desabilitar temporariamente o teste problemático:** Se a configuração do banco de dados for complexa ou exigir um ambiente externo, desabilitar temporariamente o teste `AgendamentoApplicationTests` pode permitir que o projeto compile e que a análise do código continue. Isso não é uma solução definitiva, mas um paliativo para avançar.

3. **Executar testes individualmente:** Tentar executar os testes de forma mais granular para isolar o problema.

Começarei verificando as configurações de teste e, se necessário, desabilitarei o teste `AgendamentoApplicationTests` para prosseguir com a compilação e a análise completa do projeto.

31. Conclusão da Análise Detalhada

A análise aprofundada do projeto `agendamento` revelou uma base sólida, com a utilização de tecnologias modernas como Spring Boot 3.5.3, Java 21, Maven, Spring Data JPA, MapStruct e Lombok. A estrutura em camadas (controller, service, repository, model, dto, exception) é bem definida e segue padrões comuns de desenvolvimento de aplicações Spring Boot.

Pontos Fortes Identificados:

- **Uso de Tecnologias Atuais:** A escolha de Java 21 e Spring Boot 3.5.3 demonstra uma preocupação com a modernidade e o suporte a longo prazo.
- **Spring Data JPA:** Simplifica a camada de persistência e permite a criação de consultas declarativas.
- **MapStruct:** Garante mapeamento de objetos eficiente e seguro em tempo de compilação.
- **Lombok:** Reduz a verbosidade do código, tornando-o mais limpo e conciso.
- **Records para DTOs:** Excelente para DTOs, promovendo imutabilidade e concisão.
- **Validações Declarativas:** O uso de `@Valid` e anotações de validação nos DTOs centraliza e automatiza a validação de entrada.
- **Tratamento Centralizado de Exceções:** O `GlobalExceptionHandler` é uma boa prática para padronizar as respostas de erro da API.
- **Paginação:** A implementação de paginação nos endpoints é crucial para a escalabilidade.

Oportunidades de Melhoria e Pontos Críticos (Foco para Refatoração):

- 1. Modelagem do Relacionamento Profissional com Cliente :** Este é o ponto mais crítico. O relacionamento `ManyToOne` de `Profissional` com `Cliente` (`idCliente`) e a forma como é tratado na camada de serviço sugerem que um profissional está associado a *um* cliente específico. Em um sistema de agendamento, um profissional geralmente atende *vários* clientes. É fundamental esclarecer a intenção original dessa modelagem. As opções incluem:
 - Se `idCliente` representa o *proprietário* do estabelecimento ou um cliente de referência para o profissional, o nome da propriedade e o relacionamento devem ser mais claros.
 - Se um profissional pode atender múltiplos clientes, a modelagem deve ser revista para um relacionamento `ManyToMany` ou `OneToMany` (do lado do `Cliente` para `Profissional`).
 - Se a intenção é que o `Profissional` seja um *usuário* do sistema que também pode ser um `Cliente` , uma entidade `Usuario` com papéis ou herança seria mais apropriada.**Esta refatoração é crucial para a aplicação dos princípios SOLID (especialmente SRP e OCP) e para a clareza do domínio.**
- 2. Consistência no Tratamento de Exceções:** Há uma inconsistência no tipo de exceção lançada para o cenário de “recurso não encontrado” (`BusinessException` em `ClienteServiceImpl` , `ResourceNotFoundException` em `EspecialidadeServiceImpl` , e `EntityNotFoundException` em `ProfissionalServiceImpl`). Além disso, o `GlobalExceptionHandler` não trata `ResourceNotFoundException` explicitamente, o que pode levar a respostas HTTP 500 inesperadas. **É essencial padronizar o uso de uma única exceção (preferencialmente `ResourceNotFoundException`) para recursos não encontrados e garantir que ela seja tratada corretamente no `GlobalExceptionHandler` com um status HTTP 404.**
- 3. Inconsistência no `ClienteMapper` :** O `ClienteMapper` é uma classe manual com métodos estáticos, enquanto os outros mappers utilizam interfaces `MapStruct` que estendem `BaseMapper` . Para manter a consistência e aproveitar os benefícios do `MapStruct`, o `ClienteMapper` deve ser refatorado para seguir o mesmo padrão.

4. **Nomenclatura de Métodos:** Embora consistente em português, a padronização para inglês nos nomes de métodos e variáveis é uma convenção comum em projetos Java e pode melhorar a legibilidade e a colaboração em equipes internacionais.
5. **Códigos de Status HTTP para Criação:** O `EspecialidadeController` retorna `200 OK` para operações de criação (`POST`), quando o correto seria `201 Created` para indicar que um novo recurso foi criado com sucesso. O `ProfissionalController` já faz isso corretamente.

32. Próximos Passos e Discussão com o Usuário

Com a análise concluída e o projeto compilando com sucesso, o próximo passo é apresentar esta análise ao usuário. O ponto mais importante a ser discutido é a **modelagem do relacionamento Profissional com Cliente**. A resposta do usuário a essa questão guiará as próximas refatorações e a implementação de novas funcionalidades.

Após a discussão, será proposto um plano de refatoração detalhado para abordar os pontos de melhoria identificados, com foco em Clean Code e princípios SOLID. Em seguida, as melhorias serão implementadas e o projeto será preparado para futuras expansões.

33. Esclarecimento da Modelagem de Profissional e Cliente

Com base no feedback do usuário, a intenção da modelagem é a seguinte:

- **Cliente (entidade atual):** Representa as **clínicas** que contratarão a API. Cada `Cliente` (clínica) é um *tenant* no sistema multi-tenant.
- **Profissional:** Um profissional pertence a uma **única clínica** (`idCliente` na entidade `Profissional` se refere ao ID da clínica). No entanto, um profissional pode atender *vários pacientes* dentro dessa clínica. A separação dos dados (profissionais, pacientes, agendamentos) entre as clínicas será feita através do `idCliente`.

Revisão da Observação Anterior:

A observação inicial sobre o relacionamento `ManyToOne` de `Profissional` com `Cliente` (`idCliente`) ser conceitualmente incorreto para um sistema de agendamento de serviços (onde um profissional atende muitos clientes/pacientes) foi esclarecida. Na verdade, a

entidade `Cliente` aqui representa o *contratante da API* (a clínica), e não o paciente final. Portanto, um `Profissional` pertencer a uma `Cliente` (clínica) específica via `ManyToOne` está **correto** para o cenário de multi-tenancy onde a clínica é o tenant.

Novos Pontos de Atenção e Oportunidades de Melhoria com base no Esclarecimento:

1. **Nomenclatura da Entidade `Cliente`** : A entidade `Cliente` (que representa a clínica/tenant) pode gerar confusão, pois o termo

“cliente” é frequentemente associado ao paciente final. Sugere-se renomear `Cliente` para algo como `Clinica` ou `Tenant` para maior clareza e semântica do domínio. Isso impactaria a entidade, repositório, serviço, DTOs e controladores relacionados.

1. **Separação de Pacientes:** O usuário mencionou que

“o profissional, paciente de cada clinica vai ser separado pelo idCliente” . Isso implica que haverá uma entidade `Paciente` (ainda não presente no código) que também estará associada a um `Cliente` (clínica). A modelagem precisará incluir essa nova entidade e seus relacionamentos.

1. **Multi-tenancy:** A abordagem de usar `idCliente` para separar os dados de diferentes clínicas é uma forma de implementar multi-tenancy (multi-inquilino) em um único banco de dados. Isso requer que todas as consultas e operações de persistência para `Profissional` , `Paciente` (futuro) e `Agendamento` (futuro) sempre filtrem pelo `idClinica` (antigo `idCliente`). Isso pode ser implementado de diversas formas, como:
 - **Filtros JPA:** Usar `@Filter` do Hibernate para aplicar o filtro automaticamente em todas as consultas.
 - **Contexto de Segurança:** Injetar o `idClinica` do usuário logado (que representa a clínica) em todas as operações.
 - **Camada de Serviço:** Garantir que a camada de serviço sempre inclua o `idClinica` nas operações de busca e persistência.

2. Princípios SOLID:

- **SRP (Single Responsibility Principle):** A entidade `Profissional` agora tem a responsabilidade de pertencer a uma clínica. A futura entidade `Paciente` terá a

responsabilidade de pertencer a uma clínica e ser atendido por profissionais. A renomeação de `Cliente` para `Clinica` ajudará a deixar as responsabilidades mais claras.

- **OCP (Open/Closed Principle):** A forma como o `idClinica` será propagado e utilizado nas consultas deve ser robusta para que novas entidades ou funcionalidades não exijam modificações em cascata em todo o código.

34. Plano de Refatoração Proposto (Fase 1: Ajustes de Modelagem e Nomenclatura)

Com base no esclarecimento da modelagem, proponho o seguinte plano de refatoração inicial:

1. Renomear a Entidade `Cliente` para `Clinica` (ou `Tenant`):

- Renomear a classe `Cliente.java` para `Clinica.java`.
- Atualizar o nome da tabela no `@Table(name = "clinica")`.
- Renomear `ClienteRepository` para `ClinicaRepository`.
- Renomear `ClienteService` e `ClienteServiceImpl` para `ClinicaService` e `ClinicaServiceImpl`.
- Renomear `ClienteRequestDTO` e `ClienteResponseDTO` para `ClinicaRequestDTO` e `ClinicaResponseDTO`.
- Renomear `ClienteMapper` para `ClinicaMapper` e refatorá-lo para ser uma interface `MapStruct` que estende `BaseMapper`.
- Atualizar `ClienteController` para `ClinicaController` e ajustar os endpoints e DTOs.
- Atualizar todas as referências a `Cliente` para `Clinica` em todo o projeto (entidades, serviços, controladores, mappers, etc.).

2. Ajustar o Relacionamento em `Profissional`:

- Renomear o campo `idCliente` em `Profissional.java` para `idClinica` (ou `clinica`).
- Atualizar o `@JoinColumn(name = "clinica_id")`.
- Atualizar as referências a `idCliente` para `idClinica` (ou `clinica`) nos DTOs de `Profissional` e em todas as camadas que o utilizam.

3. Padronizar o Tratamento de Exceções:

- Garantir que todas as exceções de “recurso não encontrado” lancem `ResourceNotFoundException`.
- Atualizar `GlobalExceptionHandler` para tratar `ResourceNotFoundException` com `HttpStatus.NOT_FOUND`.
- Remover o uso de `BusinessException` para cenários de “recurso não encontrado” em `ClienteServiceImpl`.

4. Padronizar Códigos de Status HTTP:

- Garantir que todos os endpoints `POST` retornem `HttpStatus.CREATED` (201).

5. Padronizar Nomenclatura de Métodos (Opcional, mas recomendado):

- Considerar refatorar os nomes de métodos de português para inglês em todas as camadas para maior consistência com as convenções da comunidade Java.

Este plano de refatoração será executado em fases, começando pelos ajustes de nomenclatura e modelagem, que são a base para as próximas melhorias. Após a conclusão desta fase, será possível discutir a implementação da entidade `Paciente` e a estratégia de multi-tenancy mais adequada.